

COMPUTER NETWORKS LAB MANUAL



Dr. Muhammad Siddique

Computer Networks Lab Manual (Cisco Packet Tracer, Wireshark, Real Hardware & Python Sockets)

Dr. Muhammad Siddique

August 2026

Contents

1	Week 1: Network Fundamentals and Connectivity Testing	2
2	Week 2: The OSI and TCP/IP Reference Models	6
3	Week 3: IPv4 Addressing and Subnetting	8
4	Week 4: Devices, Cabling, and Your First LAN in Packet Tracer	11
5	Week 5: Switch Management and VLANs	14
6	Week 6: Router Interfaces and Static Routing	17
7	Week 7: Dynamic Routing – RIPv2 and OSPF	20
8	Week 8: DHCP and NAT	23
9	Week 9: Transport Layer Analysis with Wireshark	26
10	Week 10: Application Layer Protocols in Action	29
11	Week 11: Socket Programming – Building the Transport Layer Yourself	32
12	Week 12: Network Security Fundamentals	36
13	Week 13: Final Project – Complete Enterprise Campus Network	39

1 Week 1: Network Fundamentals and Connectivity Testing

Objective: Explore your own machine’s network identity, learn the essential diagnostic commands, and adopt a systematic method for isolating connectivity faults.

Tools: Windows Command Prompt / Linux terminal; access to a live LAN.

Tasks:

1. **Task 1:** Discover your host’s complete IP configuration and explain every field it reports.
2. **Task 2:** Test reachability with `ping` against loopback, gateway, and a remote site, and interpret TTL and round-trip time.
3. **Task 3:** Trace the end-to-end path with `tracert`/`traceroute` and explain what each hop represents.
4. **Task 4:** Inspect the ARP cache and MAC address of your machine, relating Layer 2 to Layer 3 addressing.
5. **Task 5:** Diagnose three deliberately broken scenarios using the layered troubleshooting method.

Details of Lab Experiment:

Task 1: Who am I on this network?

```
C:\> ipconfig /all

Host Name . . . . . : LAB-PC-07
Physical Address. . . . : A4-BB-6D-3E-11-C2    <- MAC (
    Layer 2)
DHCP Enabled. . . . . : Yes
IPv4 Address. . . . . : 192.168.10.25        <- host
    identity (L3)
Subnet Mask . . . . . : 255.255.255.0        <- local
    network boundary
Default Gateway . . . . : 192.168.10.1        <- exit door
    off this LAN
DNS Servers . . . . . : 8.8.8.8              <- name
    resolution
Lease Obtained/Expires : dates               <- DHCP
    lease lifetime
```

Listing 1: Windows: full adapter configuration

On Linux the equivalents are `ip addr show` and `ip route`. Internalize the meaning of four fields now: the *IPv4 address* identifies this host, the *subnet mask* defines which destinations are “on my LAN” versus “beyond the gateway,” the *default gateway* is the router that forwards everything off-LAN, and *DNS* translates names into addresses. Almost every fault you will ever diagnose begins by checking these four values.

Task 2: ping – testing reachability

```
C:\> ping 127.0.0.1           :: loopback: is the TCP/IP
    stack itself alive?
C:\> ping 192.168.10.1        :: default gateway: can I reach
    my router?
C:\> ping 8.8.8.8             :: public IP: does routing work
    end to end?
C:\> ping google.com          :: name: does DNS resolution
    work?

Reply from 8.8.8.8: bytes=32 time=14ms TTL=115
Reply from 8.8.8.8: bytes=32 time=13ms TTL=115
```

Listing 2: A layered ping sequence, most-local to most-remote

This ordered sequence is a diagnostic ladder: loopback success proves your stack is installed; gateway success proves your LAN wiring, switch, and addressing are good; remote-IP success proves routing works somewhere upstream; name success finally proves DNS. Wherever the ladder first fails tells you roughly *where* the fault lives. TTL=115 means 117–128 hops were possible originally and the packet has crossed a handful of routers so far; *time* is the measured round-trip latency in milliseconds.

Task 3: tracert – following the path

1	1 ms	192.168.10.1	our LAN router (gateway)
2	4 ms	100.64.12.1	ISP aggregation router
3	9 ms	39.144.0.1	ISP core
4	13 ms	108.170.245.1	transit provider
5	14 ms	142.250.235.x	destination network edge

Listing 3: Annotated traceroute toward a public server

Each numbered hop is one router that decremented the packet’s TTL to zero and returned an ICMP “time exceeded” message – that reply mechanism is exactly how the tool maps the path. Asterisks (* * *) on a hop

mean that router simply declines to answer, which is often policy rather than failure; judge health by whether hops *after* it still respond.

Task 4: ARP – gluing Layer 3 to Layer 2

```
C:\> arp -a
```

Internet Address	Physical Address	Type
192.168.10.1	a4-bb-6d-3e-11-c2	dynamic
192.168.10.25	d8-f2-ca-77-90-11	dynamic

Listing 4: Viewing the ARP cache

IP addresses get a packet routed to the right *network*, but delivery on the local segment happens by *MAC address*; ARP is the protocol that asks “who has 192.168.10.1? tell me your MAC” and caches the answer. Clear it with `arp -d *` and ping again – you can watch the entry repopulate, which demonstrates that ARP runs silently before almost every LAN communication.

Task 5: Systematic fault isolation

Your instructor will break the lab network in three ways (unplug a cable, misconfigure a default gateway, disable the NIC). For each symptom, record which rungs of the Task-2 ladder fail, then apply the mapping below:

Failing test	Likely layer	First checks
Loopback fails	Stack/NIC driver	Reinstall adapter, reboot
Gateway ping fails	L1/L2 or IP config	Cable, link LED, mask/gateway
Remote IP fails	Routing	Gateway config, ISP link
Name fails only	DNS	<code>ipconfig /dnsflush</code> , DNS setting

Why it matters: random trial-and-error wastes hours; the layered ladder converges on most faults in under five minutes, and employers consistently rank structured troubleshooting above configuration memorization.

Summary:

You learned to read your host’s complete network identity, tested reachability in a disciplined bottom-up order, mapped a real Internet path, observed ARP bridging IP to MAC, and applied a layered method to localize injected faults. Every subsequent lab assumes you can do all five fluently.

Review Questions:

1. A colleague's PC cannot open websites, yet `ping 8.8.8.8` succeeds. Which specific subsystem is suspect, and which command confirms it?
2. Why does TTL decrease by (approximately) one per router, and why is it "approximately"?
3. Distinguish a MAC address from an IP address in purpose, scope of validity, and protocol layer.

2 Week 2: The OSI and TCP/IP Reference Models

Objective: Master the layered view of networking – the shared vocabulary of the entire industry – and observe encapsulation happening in a real capture.

Tools: Wireshark; browser; ping.

Tasks:

1. **Task 1:** Recite the seven OSI layers bottom-up with a one-line function for each, from memory.
2. **Task 2:** Map the TCP/IP model onto OSI, and place given protocols, PDUs, and devices at their layers.
3. **Task 3:** Explain encapsulation and de-encapsulation, naming what each layer adds and strips.
4. **Task 4:** Capture live traffic in Wireshark and identify the stacked headers of a single ping.
5. **Task 5:** Classify ten everyday activities into their dominant layer(s).

Details of Lab Experiment:

Tasks 1–2: The layered reference tables

#	Layer	Core question it answers	PDU	Examples / devices
7	Application	What service?	Data	HTTP, DNS, SMTP
6	Presentation	What format?	Data	TLS, JPEG, encoding
5	Session	Whose conversation?	Data	NetBIOS, RPC
4	Transport	Which process?	Segment	TCP, UDP, ports
3	Network	Which network?	Packet	IP, ICMP; routers
2	Data Link	Which device on segment?	Frame	Ethernet; switches
1	Physical	Which wire/signal?	Bits	Copper, fiber, hubs

The TCP/IP model compresses layers 5–6 into Application and keeps Physical/Data-Link as the single Network Access layer. Memorize the mnemonic for OSI bottom-up – *Please Do Not Throw Sausage Pizza Away* – and, more importantly, the *question each layer answers*, because that question is how you troubleshoot (each failed Task-1 test last week localized a broken *layer*).

Task 3: Encapsulation – envelopes inside envelopes

When you send data, each layer wraps the layer above:

```
[ Eth hdr | IP hdr | TCP hdr | Application data | Eth ftr ]
```

The sender's stack adds headers downward; every intermediate router peels only up to Layer 3, rewrites Layer 2 framing, and re-encapsulates toward the next hop; the receiver strips upward until the application receives pure payload. Two consequences are worth stating explicitly: *MAC addresses change on every hop* while source/destination IPs stay constant end-to-end, and each header consumes bandwidth, which is why MTU and overhead matter in capacity planning.

Tasks 4–5: Seeing the layers in Wireshark

Start a capture filtered on ICMP, then from a terminal run `ping 8.8.8.8`. Stop the capture and click any echo-request frame; Wireshark's middle pane shows exactly the nested structure of Task 3 – expandable **Ethernet II**, then **Internet Protocol Version 4**, then **Internet Control Message Protocol**. Record for your report: source/destination MAC, source/destination IP, ICMP type/code, and confirm the MACs belong to you and your gateway while the IPs belong to you and the far server. Then classify: video call, downloading a driver, choosing Wi-Fi channel, a switch forwarding by MAC, a router choosing next-hop, ZIP compression, a three-way conversation handshake – argue the layer for each and defend one debatable case.

Summary:

You can now name all seven layers with their functions and PDUs, map them onto the pragmatic TCP/IP stack, narrate encapsulation down and up a path, and point at those very headers inside a live capture. The rest of this course climbs the stack week by week, and this vocabulary is the map for the climb.

Review Questions:

1. A switch forwards by destination MAC and a router by destination IP – state the layer and PDU each examines.
2. During one hop, which header fields are rewritten and which survive unchanged? Why?
3. Why do textbooks say OSI is the “teaching model” while TCP/IP is the “running model”?

3 Week 3: IPv4 Addressing and Subnetting

Objective: Read any IPv4 address fluently – its class, scope, network portion, and host capacity – and design real addressing plans with subnetting and VLSM.

Tools: Calculator, paper; Packet Tracer (optional verification).

Tasks:

1. **Task 1:** Classify addresses by class and identify public vs private ranges instantly.
2. **Task 2:** Convert dotted-decimal to binary and back by hand for arbitrary octets.
3. **Task 3:** Given IP + prefix, compute network address, broadcast, first/last host, and capacity.
4. **Task 4:** Split a /24 into four equal subnets and tabulate all ranges.
5. **Task 5:** Design a complete VLSM plan for a three-department company plus WAN links.

Details of Lab Experiment:

Tasks 1–2: Classes and binary arithmetic

Private (RFC 1918) ranges – routable only inside organizations, translated at the edge:

Class	Range	Default mask
A	1.0.0.0 – 126.255.255.255	255.0.0.0 (/8)
B	128.0.0.0 – 191.255.255.255	255.255.0.0 (/16)
C	192.0.0.0 – 223.255.255.255	255.255.255.0 (/24)
D / E	224–239 multicast / 240–255 experimental	—

Private use only
10.0.0.0/8
172.16.0.0/12
192.168.0.0/16
127.0.0.0/8 loopback

Binary conversion uses place values 128, 64, 32, 16, 8, 4, 2, 1 per octet: 192 = 11000000, 10 = 00001010. Practice until converting 172.31.255.200 takes under thirty seconds – every skill in Weeks 4–13 quietly depends on this fluency.

Task 3: Dissecting an address

Given 192.168.10.130/26: the mask borrows 2 bits into the host field, leaving $2^6 - 2 = 62$ usable hosts per subnet with block size $256 - 192 = 64$. Subnets therefore begin at .0, .64, .128, .192; ours (.130) falls in the third:

Network 192.168.10.128 Hosts .129–.190 Broadcast .191

The two formulas to memorize:

$$\# \text{subnets} = 2^{\text{borrowed bits}}, \quad \# \text{usable hosts} = 2^{\text{host bits}} - 2$$

Task 4: Equal-sized subnetting

Split 192.168.10.0/24 into four equal subnets ($2^2 = 4$, borrow 2 bits $\Rightarrow$ /26):

Subnet	Network	First host	Last host	Broadcast
0	192.168.10.0	.1	.62	.63
1	192.168.10.64	.65	.126	.127
2	192.168.10.128	.129	.190	.191
3	192.168.10.192	.193	.254	.255

Verify each row by hand once, then confirm in Packet Tracer by assigning .65 and .66 to two PCs and pinging across.

Task 5: VLSM – variable-length allocation

Requirement: Department A needs 100 hosts, B needs 50, C needs 20, and one WAN link needs 2 – all carved from 192.168.1.0/24. Rule one: *allocate largest-first* so boundaries never collide:

Block	Prefix	Network	Usable range	Bcast	Cap.
Dept A (100)	/25	192.168.1.0	.1 – .126	.127	126
Dept B (50)	/26	192.168.1.128	.129 – .190	.191	62
Dept C (20)	/27	192.168.1.192	.193 – .222	.223	30
WAN (2)	/30	192.168.1.224	.225 – .226	.227	2

Notice the discipline: each new block starts immediately after the previous broadcast, sizes double downward ($/25 \rightarrow /26 \rightarrow /27 \rightarrow /30$), and every department keeps growth headroom. Redo the same design from 172.20.0.0/16 for departments of 500, 120, and 60 hosts plus four WAN links – bring the finished table to next week’s lab.

Summary:

You can now classify any IPv4 address on sight, convert octets to binary mentally, dissect an address into network/host/broadcast, and produce both equal-size and VLSM addressing plans – the exact prerequisite for every routing and switching configuration that follows.

Review Questions:

1. Is 172.32.5.9 private? Justify using the /12 boundary.
2. How many /29 subnets fit inside one /24, and how many hosts survive per subnet?
3. Why does VLSM mandate largest-first ordering? Give a concrete collision example if violated.

4 Week 4: Devices, Cabling, and Your First LAN in Packet Tracer

Objective: Know what each network device fundamentally does, select correct cabling, and build and verify a working LAN from scratch in Cisco Packet Tracer.

Tools: Cisco Packet Tracer.

Tasks:

1. **Task 1:** Contrast hub, switch, and router by layer, intelligence, and domains affected.
2. **Task 2:** Choose correctly among straight-through, crossover, and rollover cables for given device pairs.
3. **Task 3:** Build a two-PC + switch LAN and assign addressing from your Week-3 plan.
4. **Task 4:** Verify connectivity with ping and inspect the switch's learned MAC table.
5. **Task 5:** Extend the LAN with a router and count collision vs broadcast domains.

Details of Lab Experiment:

Task 1: The device zoo

- **Hub (Layer 1):** electrically repeats every bit to *every* port – one shared collision domain, zero intelligence. Obsolete but pedagogically vital.
- **Switch (Layer 2):** forwards frames by destination MAC using a dynamically learned MAC address table; *each port is its own collision domain*, while the whole switch remains one broadcast domain.
- **Router (Layer 3):** routes packets by destination IP between networks; each interface terminates one broadcast domain – routers are the firebreaks that stop broadcasts.

Task 2: Copper cable selection

Cable	Connects	Rule of thumb
Straight-through	unlike devices: PC→switch, switch→router	“different families”
Crossover	like devices: PC→PC, switch→switch, router→router	“same family”
Rollover (console)	PC COM port → device console	blue Cisco cable

Modern gigabit ports implement *Auto-MDIX* and electronically swap pairs when wrong, so mis-cabling often still links – but configure deliberately, and know why it works.

Tasks 3–4: Build and verify

Place 2960 switch and two PCs; connect each PC to **FastEthernet0/1** and **0/2** with straight-through copper. Per PC: *Desktop* → *IP Configuration* → static 192.168.10.11/24 and 192.168.10.12/24 (gateway .1 reserved for later). Then verify:

```
C:\> ping 192.168.10.12
Reply from 192.168.10.12: bytes=32 time<1ms TTL=128

C:\> arp -a
192.168.10.12    d8-f2-ca-77-90-12    dynamic
```

Listing 5: Verification from PC1’s command prompt

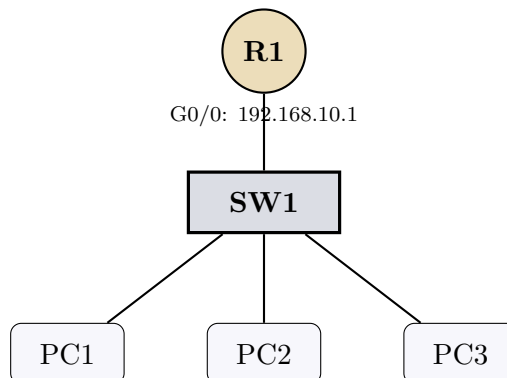
On the switch CLI (*CLI* tab), watch learning happen:

```
SW1> enable
SW1# show mac address-table

Mac Address Table
-----
Vlan    Mac Address      Type      Ports
----
1       d8f2.ca77.9011   DYNAMIC   Fa0/1
1       d8f2.ca77.9012   DYNAMIC   Fa0/2
```

Listing 6: The transparent bridge at work

Unknown-unicast behavior is worth demonstrating: temporarily clear the table (`clear mac address-table dynamic`), ping, and note the switch floods the first frame out *every* port before learning pins down the right one.

Task 5: Domains and the router edge

Attach R1's G0/0 at 192.168.10.1/24 (no **shutdown!**) and count together: **four collision domains** (every switch port plus the router link share none) but still **one broadcast domain** – broadcasts from PC1 annoy everyone including the router. Adding a second router interface tomorrow would create the second broadcast domain; that boundary-hungry property is precisely what makes routers the backbone of the Internet.

Summary:

You selected media intelligently, built a functioning switched LAN, watched a real MAC table learn and flood, and quantified collision versus broadcast domains. Next week we teach the switch to segregate users into VLANs – splitting that single broadcast domain on purpose.

Review Questions:

1. Why did the very first ping after boot sometimes lose one packet, and what fixed it?
2. With 12 PCs on one hub versus on one switch, compute collision domains in each case and explain the performance implication.
3. Which single command proved the switch had learned both PCs' locations?

5 Week 5: Switch Management and VLANs

Objective: Manage a switch as a managed device, then partition one physical switch into isolated virtual LANs and trunk them between switches.

Tools: Cisco Packet Tracer (two 2960 switches, four PCs).

Tasks:

1. **Task 1:** Apply baseline management: hostname, console/enable passwords, encrypted storage, legal banner.
2. **Task 2:** Give the switch an SVI management IP and default gateway so it becomes remotely reachable.
3. **Task 3:** Create VLANs 10/20/99 and assign access ports per a written plan.
4. **Task 4:** Prove inter-VLAN isolation and intra-VLAN reachability with pings.
5. **Task 5:** Configure an 802.1Q trunk between the two switches and verify VLAN traffic crosses it correctly.

Details of Lab Experiment:

Tasks 1–2: Baseline management configuration

IOS navigation matters: *user EXEC* (Switch>) → *enable* → *privileged EXEC* (Switch#) → *configure terminal* → *global config* → *interface/line* sub-modes; *exit* retreats one level, *end* or *Ctrl-Z* returns to privileged EXEC.

```
Switch> enable
Switch# configure terminal
Switch(config)# hostname SW1
SW1(config)# enable secret Cisco@123
SW1(config)# line console 0
SW1(config-line)# password Cons@123
SW1(config-line)# login
SW1(config-line)# exit
SW1(config)# service password-encryption
SW1(config)# banner motd #Authorized Personnel Only!#
SW1(config)# vlan 99
SW1(config-vlan)# name MANAGEMENT
SW1(config-vlan)# interface vlan 99
SW1(config-if)# ip address 192.168.99.2 255.255.255.0
SW1(config-if)# no shutdown
SW1(config-if)# exit
```

```
SW1(config)# ip default-gateway 192.168.99.1
SW1(config)# end
SW1# copy running-config startup-config
```

Listing 7: Baseline hardening and management SVI

Two classic traps: `enable secret` is hashed while plain `enable password` is trivially reversible – always the former; and without `copy running-config startup-config` (or `write memory`) everything vanishes on reload. The SVI is how you `ping`/Telnet the switch itself over the network; the default-gateway line lets those replies leave the local subnet.

Tasks 3–4: VLAN creation and assignment

VLAN	Name	Purpose / ports
10	STAFF	Fa0/2–Fa0/10
20	STUDENTS	Fa0/11–Fa0/20
99	MANAGEMENT	SVI only

```
SW1(config)# vlan 10
SW1(config-vlan)# name STAFF
SW1(config-vlan)# vlan 20
SW1(config-vlan)# name STUDENTS
SW1(config-vlan)# exit
SW1(config)# interface range fa0/2 - 10
SW1(config-if-range)# switchport mode access
SW1(config-if-range)# switchport access vlan 10
SW1(config-if-range)# exit
SW1(config)# interface range fa0/11 - 20
SW1(config-if-range)# switchport mode access
SW1(config-if-range)# switchport access vlan 20
```

Listing 8: Creating VLANs and assigning access ports

Verification and the isolation proof:

```
SW1# show vlan brief

VLAN  Name                Status    Ports
----  -
10    STAFF                  active    Fa0/2..Fa0/10
20    STUDENTS              active    Fa0/11..Fa0/20
99    MANAGEMENT            active

PC-STAFF> ping 192.168.10.12  :: same VLAN -> replies
PC-STAFF> ping 192.168.20.11  :: other VLAN -> no
replies
```

Listing 9: Confirming the partition

Each VLAN is a separate broadcast domain living inside one box: frames never leak between VLANs at Layer 2, regardless of addressing mistakes. Cross-VLAN communication now requires a router – that is Week 6’s business.

Task 5: Trunking between switches

Access ports strip/insert no tags and serve end hosts; a *trunk* carries *multiple* VLANs simultaneously by tagging each frame with an 802.1Q VLAN ID:

```
SW1(config)# interface gi0/1
SW1(config-if)# switchport mode trunk
SW1(config-if)# switchport trunk allowed vlan 10,20,99
```

Listing 10: 802.1Q trunk on the inter-switch link, both switches

Build the mirrored lab – SW2 hosting its own STAFF/STUDENTS PCs on the same VLAN numbers – then prove tagging works: a VLAN-10 PC on SW1 pings its VLAN-10 twin on SW2 successfully, while VLAN-20 peers remain mutually unreachable even though they share the same physical cable. Run `show interfaces trunk` and read back the allowed list and active trunking protocol. **Design habit from day one:** always prune `allowed vlan` to exactly the VLANs a link must carry, and reserve one unused native/management VLAN (our 99) – sloppy trunks become VLAN-hopping attack surface in Week 12.

Summary:

You hardened and addressed a switch, sliced it into verified VLANs, and extended those segments across a tagged trunk. One physical plant now behaves like several logical switches – the single most-used technology in every production campus network.

Review Questions:

1. Why does `switchport mode access` on user ports improve both security and convergence?
2. A frame from VLAN 20 arrives on a trunk whose allowed list omits 20 – what happens, and where?
3. What exactly does `ip default-gateway enable` on a switch, and when does the switch itself use it?

6 Week 6: Router Interfaces and Static Routing

Objective: Bring up router interfaces, understand the routing table, and build full reachability across a multi-network topology using static and default routes.

Tools: Cisco Packet Tracer (three 1941 routers, serial links, two LANs).

Tasks:

1. **Task 1:** Perform baseline router setup: hostname, banners, console/VTY protection, interface addressing.
2. **Task 2:** Interpret `show ip route` codes and distinguish connected vs static entries.
3. **Task 3:** Configure static routes on every router for non-directly-connected networks.
4. **Task 4:** Deploy a default route (stub-router design) and propagate it conceptually.
5. **Task 5:** Prove bidirectional end-to-end connectivity and explain why return paths matter.

Details of Lab Experiment:

Task 1–2: Topology and bringing interfaces up

Addressing: R1 S0/0/0=10.0.12.1/30 toward R2(.2); R2 S0/0/1=10.0.23.1/30 toward R3(.2); R1–R3 direct link 10.0.13.0/30 uses .1/.2. Each LAN hangs off G0/0.

```
R1(config)# interface g0/0
R1(config-if)# ip address 192.168.1.1 255.255.255.0
R1(config-if)# no shutdown
R1(config-if)# interface s0/0/0
R1(config-if)# ip address 10.0.12.1 255.255.255.252
R1(config-if)# clock rate 64000          :: DCE side only,
           in PT
R1(config-if)# no shutdown
```

Listing 11: Interface bring-up on R1 (repeat pattern everywhere)

Immediately inspect what IOS knows natively:

```
R1# show ip route

C    192.168.1.0/24 is directly connected,
    GigabitEthernet0/0
C    10.0.12.0/30 is directly connected, Serial0/0/0
C    10.0.13.0/30 is directly connected, Serial0/0/1
L    10.0.12.1/32 is directly connected, Serial0/0/0
```

Listing 12: Reading the routing table – codes tell the story

C = connected, L = local interface address. Everything *absent* is currently unreachable: ping 192.168.2.1 from R1 and it fails – not because the wire is bad, but because R1 literally has no idea that network exists.

Task 3: Static routes – teaching each router the map

```
R1(config)# ip route 192.168.2.0 255.255.255.0 10.0.13.2
R1(config)# ip route 10.0.23.0 255.255.255.252 10.0.12.2

R3(config)# ip route 192.168.1.0 255.255.255.0 10.0.13.1
R3(config)# ip route 10.0.12.0 255.255.255.252 10.0.23.1
```

Listing 13: R1 learns distant networks; symmetry on R3

The next-hop is the *neighbor's* IP – “for that network, hand the packet to him.” Now the flagship test, and the semester’s most instructive half-failure: ping 192.168.2.11 from a LAN-A PC. If R3 lacks its return route, the echo *request* arrives but the *reply* dies – Windows reports timeouts, misleading you toward the wrong end. Always ask both directions of the question: forward *and* return.

Task 4: Default routes for stub edges

R1 owns exactly one sensible exit toward “everything else” through R2/R3; a default route replaces many specific statements:

```
R1(config)# ip route 0.0.0.0 0.0.0.0 10.0.12.2
R1# show ip route | include 0.0.0.0
S*    0.0.0.0/0 [1/0] via 10.0.12.2
```

Listing 14: Stub-router default route

The quad-zero matches every destination not matched more-specifically – the longest-prefix-match rule guarantees specifics still win. This is the same construct your home router calls “default gateway.”

Task 5: Administrative distance sanity check

If both static and RIP knew 192.168.2.0, IOS installs the static: lower administrative distance wins ties between sources.

Source	AD	Source	AD
Connected	0	OSPF	110
Static	1	RIP	120
Internal EIGRP	90	External EIGRP	170

Delete one static (no `ip route ...`), watch the route vanish from the table, restore it, and save everyone’s work with `copy run start` on all three routers.

Summary:

You configured interfaces into existence, read the routing table like a map legend, supplied missing knowledge with static and default routes, and learned the bidirectional-path lesson that saves hours of false blaming. Hand-maintaining maps scales poorly though – next week the routers introduce themselves.

Review Questions:

1. PC-A pings PC-B; requests visibly arrive (you sniff them) but replies never leave. Which router(s) lack what?
2. Why is [1/0] shown beside a static route – decode both numbers.
3. When is a floating static (raised AD) useful, and write the syntax for AD 90.

7 Week 7: Dynamic Routing – RIPv2 and OSPF

Objective: Replace hand-maintained routes with self-learning protocols, compare distance-vector with link-state behavior, and watch a network heal around a dead link.

Tools: Cisco Packet Tracer (reuse Week-6 topology).

Tasks:

1. **Task 1:** Argue the operational cost of static routing at scale and define what a routing protocol automates.
2. **Task 2:** Enable RIPv2 across all three routers and verify learned routes.
3. **Task 3:** Migrate to single-area OSPF and verify adjacency plus learned routes.
4. **Task 4:** Decode metric differences (hop count vs cost) visible in the routing table.
5. **Task 5:** Break a link and measure convergence behavior under each protocol.

Details of Lab Experiment:

Task 2: RIPv2 – distance vector in action

```
R1(config)# router rip
R1(config-router)# version 2
R1(config-router)# no auto-summary
R1(config-router)# network 192.168.1.0
R1(config-router)# network 10.0.0.0
```

Listing 15: Enable RIPv2 on R1 (repeat on R2, R3 with their networks)

network has two effects – advertise those attached nets, and speak RIP out those interfaces – and uses classful boundaries, hence 10.0.0.0 covering all three /30s. Version 2 plus **no auto-summary** preserves your VLSM prefixes.

Verification:

```
R1# show ip route rip
R    192.168.2.0/24 [120/1] via 10.0.13.2, Serial0/0/1
R    10.0.23.0/30 [120/1] via 10.0.12.2, Serial0/0/0
```

Listing 16: RIP-learned route, decoded

[120/1] reads: administrative distance 120, metric 1 hop. Distance-vector logic is gossip-based – “tell your neighbors everything you know, trust what they say” – and RIP caps that trust at 15 hops.

Task 3: OSPF – link state takes over

Remove RIP cleanly (no `router rip` on every router), then:

```
R1(config)# router ospf 1
R1(config-router)# router-id 1.1.1.1
R1(config-router)# network 192.168.1.0 0.0.0.255 area 0
R1(config-router)# network 10.0.12.0 0.0.0.3 area 0
R1(config-router)# network 10.0.13.0 0.0.0.3 area 0
```

Listing 17: Single-area OSPF on R1

Wildcards are inverted masks (0.0.0.3 matches exactly the /30). Within seconds each router floods its link-state advertisements, builds an identical map of the whole area, and runs Dijkstra:

```
R1# show ip route ospf
0    192.168.2.0/24 [110/65] via 10.0.13.2, Serial0/0/1
0    10.0.23.0/30   [110/65] via 10.0.13.2

R1# show ip ospf neighbor
Neighbor ID Pri State Dead Time Address
Interface
2.2.2.2      1 FULL/DR 00:00:36 10.0.12.2 Serial0
/0/0
3.3.3.3      1 FULL/- 00:00:38 10.0.13.2 Serial0
/0/1
```

Listing 18: OSPF results and neighbors

FULL is the healthy adjacency state; cost defaults to 10^8 /bandwidth, hence 65 on serial.

Tasks 4–5: Metrics compared, failure survived

Same destination, two philosophies: RIP says “1 hop away”; OSPF says “cost 65 of aggregate bandwidth.” Shut R1’s S0/0/1 mid-lab (`interface s0/0/1: shutdown`) while pinging 192.168.2.11 from LAN A continuously:

```
t=0s    link administratively down on R1
t=1-3s  OSPF detects carrier loss, floods new LSU
t=~4s   routes recomputed; traffic flows via R1->R2->R3
        (RIP would age routes out over ~3 minutes!)
```

Listing 19: Observed convergence timeline (record your own seconds)

Restore the interface and watch the old path return. That self-healing – invisible to users, automatic, correct – is the entire justification for dynamic routing, and OSPF's fast link-state reconvergence is why it dominates enterprise cores.

Summary:

You automated the routing table twice, contrasted gossip-with-hop-counts against full-map-with-costs, and personally witnessed a network reroute around a severed link. You also met wildcard masks and adjacency states you will configure weekly in industry.

Review Questions:

1. Why does `network 10.0.0.0` suffice for three distinct /30s under RIP but not the equivalent sloppiness under OSPF?
2. Decode [110/65] completely – name both numbers' meanings and origins.
3. A router sees the same prefix from OSPF and RIP simultaneously: which installs, and what mechanism decides?

8 Week 8: DHCP and NAT

Objective: Automate host addressing with DHCP on a router, then solve IPv4 exhaustion the way the real Internet does – with Network Address Translation.

Tools: Cisco Packet Tracer (one 2911 router, switch, LAN PCs, simulated ISP/cloud).

Tasks:

1. **Task 1:** Configure an exclusive-range DHCP pool and obtain leases on client PCs.
2. **Task 2:** Explain the DORA exchange and capture it conceptually (discover, offer, request, ack).
3. **Task 3:** Motivate NAT by contrasting private RFC-1918 addressing with public scarcity.
4. **Task 4:** Implement PAT (NAT overload) toward one public interface.
5. **Task 5:** Verify translations and articulate inside-local/global terminology precisely.

Details of Lab Experiment:

Task 1–2: DHCP server on the router

```
R1(config)# ip dhcp excluded-address 192.168.10.1
192.168.10.10
R1(config)# ip dhcp pool STAFF
R1(dhcp-config)# network 192.168.10.0 255.255.255.0
R1(dhcp-config)# default-router 192.168.10.1
R1(dhcp-config)# dns-server 8.8.8.8
R1(dhcp-config)# lease 7
```

Listing 20: Pool definition with exclusions for infrastructure

Excluding `.1--.10` keeps gateways, printers, and statically-set servers out of the lease pool – forgetting this produces the classic duplicate-address mystery. Set each PC to DHCP (*IP Configuration* → DHCP); it should pull `.11` upward with gateway and DNS stamped in. The protocol dance behind the scenes: the client broadcasts *DISCOVER* (it has no address yet!), servers *OFFER* leases, the client broadcasts *REQUEST* accepting one, and the chosen server *ACKs* – four packets, all before the first web page.

```

R1# show ip dhcp binding
IP Address      Client-ID/ Hardware address      Lease
expiration
192.168.10.11   0001.C7B4.2201                   -- infinite
--
192.168.10.12   000D.BD3B.7740                   -- infinite
--

```

Listing 21: Server-side verification

Tasks 3–5: NAT/PAT – many privates, one public

Motivation first: RFC-1918 space (Week 3) is reusable by everyone but unroutable on the Internet; NAT rewrites addresses at the border so hundreds of hosts share one public IP, distinguished simultaneously by port numbers – hence *Port Address Translation*.

```

R1(config)# access-list 1 permit 192.168.10.0 0.0.0.255
R1(config)# interface g0/0
R1(config-if)# ip nat inside
R1(config-if)# interface s0/0/0
R1(config-if)# ip nat outside
R1(config-if)# exit
R1(config)# ip nat inside source list 1 interface s0/0/0
overload

```

Listing 22: PAT configuration on the border router

Generate traffic (PCs browse the simulated server at 209.165.200.5), then read the translation brain:

```

R1# show ip nat translations
Pro  Inside global      Inside local      Outside local
Outside global
tcp  203.0.113.1:1025    192.168.10.11:1025 209.165.200.5:80
209.165.200.5:80
tcp  203.0.113.1:1026    192.168.10.12:1035 209.165.200.5:80
209.165.200.5:80

```

Listing 23: Inside local vs inside global, live

Read one row aloud correctly and you own the concept: *inside local* is our host's true private identity; *inside global* is the public face it borrows – identical IPs, differentiating ports. **Discussion point for the report:** PAT saved IPv4 for decades but broke the end-to-end principle – outside hosts cannot initiate inbound connections – which is precisely the gap IPv6 (Week 13) closes permanently.

Summary:

You deployed DHCP with sane exclusions, narrated DORA, justified NAT economically, implemented overload PAT, and decoded a live translation table using flawless terminology. Nearly every network you will ever administer leans on both technologies daily.

Review Questions:

1. Why must DISCOVER and REQUEST be broadcasts while OFFER/ACK could technically be unicast?
2. Two inside hosts contact the same website from one public IP – how does the router demultiplex returning packets?
3. What breaks if you forget `ip nat outside` on the WAN interface, and how would `show ip nat statistics` reveal it?

9 Week 9: Transport Layer Analysis with Wireshark

Objective: Watch TCP and UDP perform their contrasting jobs in live captures, master the three-way handshake, and navigate captures with display filters.

Tools: Wireshark; browser; nslookup.

Tasks:

1. **Task 1:** Differentiate capture filters from display filters and apply each once.
2. **Task 2:** Capture a full web transaction and isolate the TCP handshake from the HTTP dialogue.
3. **Task 3:** Annotate sequence/acknowledgment numbers proving reliable byte-stream delivery.
4. **Task 4:** Capture a DNS query-response as the UDP counterexample and contrast header fields.
5. **Task 5:** Map the well-known port landscape and justify each protocol's transport choice.

Details of Lab Experiment:

Task 1: Two filter worlds

Capture filters (BPF syntax, decided *before* capture, conserve disk/CPU): start Wireshark with `tcp port 80` or `udp port 53`. Display filters (Wireshark syntax, applied *after*, infinitely refinable):

```
tcp.port == 80
dns
http.request.method == "GET"
tcp.flags.syn == 1 && tcp.flags.ack == 0      :: opening
    SYNs
ip.addr == 192.168.10.25                       :: one host's
    chatter
```

Listing 24: Display filters you will use constantly

Tasks 2–3: The handshake and reliability made visible

Begin capture, visit `http://example.com`, stop capture, apply `tcp.port == 80`. The story reads top-down:

No.	Source	Destination	Protocol	Info
1	192.168.10.25	93.184.x.x	TCP	49213 -> 80 [SYN] Seq=0
2	93.184.x.x	192.168.10.25	TCP	80 -> 49213 [SYN, ACK] Seq=0 Ack=1
3	192.168.10.25	93.184.x.x	TCP	49213 -> 80 [ACK] Seq=1 Ack=1
4	192.168.10.25	93.184.x.x	HTTP	GET / HTTP/1.1
5	93.184.x.x	192.168.10.25	TCP	[ACK] Seq=1 Ack =282
6	93.184.x.x	192.168.10.25	HTTP/1.1	200 OK (data follows)

Listing 25: Connection establishment, then request/response

Frames 1–3 are the celebrated three-way handshake: client proposes (SYN), server agrees-and-proposes-back (SYN, ACK), client confirms (ACK) – synchronizing initial sequence numbers so both sides can thereafter acknowledge every byte. Follow the TCP stream (right-click → *Follow*) to see the whole application conversation reassembled; notice each data frame’s acknowledgment number marching forward – that bookkeeping *is* reliability, and missing acknowledgments trigger retransmission invisibly above.

Task 4: UDP – connectionless by design

Apply the `dns` filter after running `nslookup example.com`. Exactly two packets: query out, answer back. No handshake, no acknowledgments, no sequence numbers – a 12-field-slimmer header trading reliability for speed. DNS tolerates this because it retries cheaply at the application layer; a dropped query simply gets asked again. Open both headers side-by-side in your report and tabulate what TCP has that UDP lacks.

Task 5: The port landscape

Service	Port/Proto	Service	Port/Proto
FTP data/control	20/21 TCP	HTTP	80 TCP
SSH (secure shell)	22 TCP	POP3	110 TCP
Telnet (insecure!)	23 TCP	IMAP	143 TCP
SMTP (send mail)	25 TCP	HTTPS	443 TCP
DNS	53 UDP/TCP	RDP	3389 TCP
DHCP server/client	67/68 UDP	SNMP	161 UDP

Ports multiplex thousands of conversations over one IP – your browser’s ephemeral port (49xxx above) pairs with the server’s well-known port to form a unique socket. Defend in your report: why DNS chose UDP, why mail transfer chose TCP, and why Telnet (plaintext passwords, visible in any capture!) must die in favor of SSH – foreshadowing Week 12.

Summary:

You captured and decoded both transports in the wild, walked the handshake byte-by-byte, watched acknowledgments enforce reliability, and framed the TCP-vs-UDP decision as an engineering trade-off rather than dogma. Applications next week sit exactly on the ports you now recognize on sight.

Review Questions:

1. In frame 2, whose sequence number is 0 and whose acknowledgment is 1 – and what does Ack=1 literally count?
2. A gamer complains UDP is “unreliable.” Argue why game designers choose it anyway.
3. Why can two browsers on one PC download from the same website concurrently despite sharing one source IP?

10 Week 10: Application Layer Protocols in Action

Objective: Operate and dissect the protocols users actually feel – web, names, files, and mail – correlating each wire-level dialogue with its human-visible behavior.

Tools: Packet Tracer servers; browser developer tools; Wireshark.

Tasks:

1. **Task 1:** Parse an HTTP GET/Response pair, header by header, and relate status classes to outcomes.
2. **Task 2:** Walk a DNS lookup through the hierarchy and prove caching behavior.
3. **Task 3:** Conduct an FTP session and distinguish control from data connections.
4. **Task 4:** Send and retrieve email through simulated SMTP and POP3 services.
5. **Task 5:** Correlate every protocol above against its Week-9 capture signature.

Details of Lab Experiment:

Task 1: HTTP anatomy

In Packet Tracer, deploy a **Server** with *HTTP* enabled and a PC browsing to its IP. Simultaneously inspect the exchange (PT Simulation mode, or Wireshark against a real site):

GET /index.html HTTP/1.1	<- method, resource,
version	
Host: www.example.com	<- virtual-host routing
key!	
User-Agent: Mozilla/5.0 ...	
Accept: text/html	
HTTP/1.1 200 OK	<- status class 2xx =
success	
Content-Type: text/html	<- how to interpret body
Content-Length: 1218	<- bytes following
Cache-Control: max-age=3600	<- client may reuse me

Listing 26: Request then response, headers that matter bolded in your report

Status taxonomy: 2xx success, 3xx redirect, 4xx client’s fault (404 unknown path), 5xx server’s fault. Deliberately fetch `/missing.html` and witness the 404 on the wire; note HTTP is stateless – every request stands alone, which is why cookies exist to re-attach identity.

Task 2: DNS – the Internet’s phone book

```
C:\> nslookup www.iub.edu.pk
Server: resolver.isp.net           :: your stub resolver
Non-authoritative answer:         :: came from someone's
    cache!
Address: 111.68.99.26

C:\> ipconfig /displaydns          :: proof it is cached
    locally
```

Listing 27: Recursive lookup, then cached speedup

Trace the hierarchy in your report: stub resolver → root (“ask .pk”) → .pk TLD (“ask the university’s NS”) → authoritative server (final record), all over UDP/53 from Week 9. Flush with `ipconfig /flushdns`, resolve again, and contrast timing – caching converts repeated global journeys into local lookups, and TTL dictates cache politeness.

Task 3: FTP – the two-channel oddball

From a PC command prompt against the PT FTP server (user: cisco / pass: cisco):

```
ftp> open 192.168.10.50
User: cisco Password: ****
ftp> dir                               :: control channel (21):
    listing
ftp> get config.txt                    :: NEW data channel opens
    for transfer
ftp> put backup.txt                     :: upload likewise rides
    its own channel
ftp> bye
```

Listing 28: Interactive FTP session

Commands trickle over port 21 forever, but every file spawns a separate data connection (20 or negotiated passive port) – architecturally fascinating and firewall-hostile, which is why SFTP over SSH has displaced it. Capture the session and highlight credentials crossing in cleartext: your first *observed* insecurity, not merely a warned-about one.

Task 4: Electronic mail’s relay race

Configure PT’s **Server** for SMTP+POP3, register two users, then from a client *E-Mail* app: compose to the second user, receive on the other. The journey: sender’s UA hands to his SMTP server → SMTP relays server-to-server (DNS MX records choose the destination!) → recipient pulls with POP3 (download-centric) or IMAP (server-synced). Sniff both legs and annotate: SMTP carries the message *toward* the mailbox; POP3/IMAP carry it *from* the mailbox to the reader – three protocols, one experience.

Summary:

You operated the four application pillars and grounded each in exact packet behavior – headers, hierarchies, dual channels, relay chains – closing the loop from Week 9’s transport signatures to the services users name when they say “the Internet is down.”

Review Questions:

1. Why is the HTTP **Host:** header mandatory in modern practice even though HTTP/1.0 omitted it?
2. “Non-authoritative answer” – who answered, and why is that usually fine?
3. List the FTP security sins in order of severity, and name the modern replacement for each.

11 Week 11: Socket Programming – Building the Transport Layer Yourself

Objective: Step from consuming protocols to implementing them: write working TCP and UDP applications in Python and connect the code to Weeks 9–10 theory.

Tools: Python 3 (IDLE/VS Code); two machines or VMs on one LAN.

Tasks:

1. **Task 1:** Write a TCP echo server exercising `socket/bind/listen/accept`.
2. **Task 2:** Write its client and verify bidirectional echo across the lab LAN.
3. **Task 3:** Convert the design to UDP and articulate every semantic difference the API forces.
4. **Task 4:** Grow the TCP server into a multi-client chat room using threads.
5. **Task 5:** Capture your own protocol in Wireshark and identify its fingerprint.

Details of Lab Experiment:

Tasks 1–2: TCP echo pair

```
import socket

HOST, PORT = "0.0.0.0", 5000          # all interfaces,
                                     port 5000

srv = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
# TCP
srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
srv.bind((HOST, PORT))               # claim the doorbell
srv.listen(5)                         # backlog queue of 5
print(f"Echo server listening on {PORT}")

while True:
    conn, addr = srv.accept()         # blocking: waits
    # for SYN!
    print("Connected:", addr)
    with conn:
        while True:
            data = conn.recv(1024)    # returns b"" when
            # peer closes
```

```

        if not data:
            break
        conn.sendall(b"ECHO: " + data)

```

Listing 29: tcp_echo_server.py

```

import socket

HOST, PORT = "192.168.10.20", 5000      # server's IP from
                                         Week 4!

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as
c:
    c.connect((HOST, PORT))              # performs the 3-way
                                         handshake
    while True:
        msg = input("Message (empty quits): ")
        if not msg:
            break
        c.sendall(msg.encode())
        print("Got:", c.recv(1024).decode())

```

Listing 30: tcp_echo_client.py

Map every call to theory: `bind` claims IP:port, `listen` enables the backlog, `accept` returns *when the handshake completes* – run the client and watch the server's blocked call spring alive exactly when frames 1–3 of Week 9 finish. `recv` returning empty is TCP's polite goodbye (FIN).

Task 3: The UDP variant

```

import socket

s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)      #
UDP
s.bind(("0.0.0.0", 5000))
print("UDP server ready")

while True:
    data, addr = s.recvfrom(1024)                          # datagram + return
                                                           address
    s.sendto(b"ECHO: " + data, addr)                       # reply straight
back

```

Listing 31: udp_echo_server.py – no listeners, no connections

The client swaps `connect/send/recv` for `sendto/recvfrom` – no handshake call exists to make. Your report must state three consequences observed: faster first packet (no RTT spent on SYNs), possibility of silent

loss (nothing notices), and the server serving many clients from *one* socket with zero per-client state.

Task 4: Threaded chat room

```
import socket, threading

clients = []
lock = threading.Lock()

def handle(conn, addr):
    with lock: clients.append(conn)
    try:
        while True:
            data = conn.recv(1024)
            if not data:
                break
            with lock:
                for c in clients:
                    if c is not conn:
                        c.sendall(f"{addr}: ".encode() +
                                data)
    finally:
        with lock: clients.remove(conn)
        conn.close()

srv = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
srv.bind(("0.0.0.0", 6000)); srv.listen(10)
print("Chat server on 6000")
while True:
    conn, addr = srv.accept()
    threading.Thread(target=handle, args=(conn, addr)).start()
```

Listing 32: chat_server.py – one thread per client, broadcast to all

Test with two classmates concurrently; the lock guards the shared client list against thread races – concurrency hygiene from Programming-for-AI paying off in networking.

Task 5: Meet your own traffic

While chatting, capture with filter `tcp.port == 6000`. Find your handshake triplet, your message riding as payload, and the FIN exchanges at quit. You have now built, at micro-scale, exactly what Skype-style systems do – and you can point at yourself in a capture.

Summary:

You implemented both transports from the OS API outward, felt the connection-versus-datagram difference as concrete function calls, scaled to concurrent clients safely, and recognized your own protocol in Wireshark. The transport layer is no longer magic – it is a library you have used.

Review Questions:

1. What happens at the server if the client kills abruptly without `close()`, and which `recv`-behavior cleans it up?
2. Why must `SO_REUSEADDR` precede `bind`, and what error otherwise haunts restarts?
3. State two observable symptoms that would let you distinguish lost UDP datagrams from a crashed server.

12 Week 12: Network Security Fundamentals

Objective: Harden devices and control traffic: strong credential storage, SSH over Telnet, traffic-filtering ACLs placed correctly, and switch-port access discipline.

Tools: Cisco Packet Tracer (router, two switches, three PCs, one server).

Tasks:

1. **Task 1:** Audit a default device and enumerate every weakness aloud.
2. **Task 2:** Apply credential hardening and prove encryption of stored secrets.
3. **Task 3:** Replace Telnet with SSH and verify the encrypted session.
4. **Task 4:** Write standard and extended ACLs for a stated policy and place them optimally.
5. **Task 5:** Enforce port security with sticky learning and compare violation behaviors.

Details of Lab Experiment:

Tasks 1–2: Hardening baseline

Audit first: blank enable password, plaintext VTY access, no banner, configs readable by anyone who touches the box. Then:

```
R1(config)# hostname R1
R1(config)# enable secret Strong@Pass1
R1(config)# line console 0
R1(config-line)# password ConSole@1
R1(config-line)# login
R1(config-line)# exec-timeout 5 0          :: idle logout
R1(config-line)# logging synchronous      :: logs stop
eating typing
R1(config-line)# exit
R1(config)# service password-encryption  :: scramble
stored secrets
R1(config)# security passwords min-length 8
R1(config)# banner motd #UNAUTHORIZED ACCESS PROHIBITED#
```

Listing 33: Credential hardening checklist, applied

Verify with `show running-config`: secrets appear as irreversibly hashed `secret 5 $1$mERr$...` strings while weak passwords became reversible `password 7 gibberish` – better than nothing, but `secret` remains the standard.

Task 3: SSH – encrypted remote administration

```
R1(config)# ip domain-name ietmultan.edu.pk
R1(config)# crypto key generate rsa modulus 1024
R1(config)# username admin privilege 15 secret Admin@123
R1(config)# line vty 0 4
R1(config-line)# transport input ssh          :: telnet
                refused forever
R1(config-line)# login local
R1(config-line)# exit
R1(config)# ip ssh version 2
```

Listing 34: Full SSH enablement

Test from PC1 (`ssh -l admin 192.168.1.1`); simultaneously sniff with a colleague: Telnet would parade your password in ASCII – find a teammate’s earlier FTP capture if you doubt it – while SSH shows only unreadable ciphertext. One protocol choice eliminated an entire eavesdropping class.

Task 4: ACLs – written policy as config

Policy: “VLAN 20 (students) may reach the web server’s HTTP but nothing else on the server subnet; staff may do anything.”

```
R1(config)# ip access-list extended STUDENT-POLICY
R1(config-ext-nacl)# permit tcp 192.168.20.0 0.0.0.255
                    host 192.168.30.50 eq 80
R1(config-ext-nacl)# deny ip 192.168.20.0 0.0.0.255
                    192.168.30.0 0.0.0.255
R1(config-ext-nacl)# permit ip any any          :: else
                    students lose EVERYTHING
R1(config-ext-nacl)# interface g0/0.20
R1(config-subif)# ip access-group STUDENT-POLICY in
```

Listing 35: Named extended ACL, placed nearest the SOURCE

Three laws of ACLs: processing is top-down first-match-wins; an *invisible deny ip any any* hides at the end of every list; and placement obeys *standard near destination, extended near source* (standards match only source, so placing them early over-blocks the world). Verify with pings and browses from both VLANs, and debug hits live via `show access-lists` – each line’s match counter rising as packets die or pass.

Task 5: Port security – locking the wall jack

```
SW1(config)# interface fa0/5
SW1(config-if)# switchport mode access
SW1(config-if)# switchport port-security
SW1(config-if)# switchport port-security maximum 2
SW1(config-if)# switchport port-security mac-address
sticky
SW1(config-if)# switchport port-security violation
restrict
```

Listing 36: Sticky MAC learning with restrictive violations

Sticky learning burns the first-seen MACs into running-config automatically. Swap in a rogue laptop and watch **restrict** drop its frames while logging (**show port-security interface fa0/5** counts violations) – the office stays online, the intruder does not. Compare all three modes in a table for your report: **protect** drops silently, **restrict** drops + logs, **shutdown** err-disables the port entirely (recover via **shutdown/no shutdown**).

Summary:

You audited, hardened, encrypted remote access, translated a written security policy into correctly-placed ACLs, and physically anchored switch ports to known hardware. Defense-in-depth is not one product – it is exactly these stacked disciplines, and you now hold every layer of them.

Review Questions:

1. Why does removing the **permit ip any any** line exile students from the Internet too?
2. An attacker connects to a **shutdown**-mode secured port: enumerate exactly what they experience and what the admin must type to forgive them.
3. Why is **crypto key generate rsa** impossible before **ip domain-name**, mechanically speaking?

13 Week 13: Final Project – Complete Enterprise Campus Network

Objective: Integrate the entire semester – addressing, switching, VLANs, routing, DHCP, NAT, ACLs, SSH – into one documented, tested, defensible campus design.

Tools: Cisco Packet Tracer; your completed Weeks 3–12 notebooks.

Tasks:

1. **Task 1:** Read the business scenario and translate it into a written requirements list before touching a device.
2. **Task 2:** Produce the addressing plan (VLSM) and topology drawing; defend both to your instructor.
3. **Task 3:** Implement VLANs, trunking, router-on-a-stick inter-VLAN routing, DHCP, OSPF, NAT, and SSH.
4. **Task 4:** Execute the full verification checklist and capture evidence of each pass.
5. **Task 5:** Submit documentation and demonstrate fault recovery live.

Details of Lab Experiment:

Task 1: The scenario

A growing college requires: a Staff LAN (60 hosts), a Students LAN (120 hosts), a Servers LAN (5 static hosts – web, DNS, mail), and a Management VLAN – all on private space, all Internet-capable through one public IP, students barred from reaching servers except HTTP, all administration exclusively via SSH, and wireless may substitute for one wired LAN if demonstrated with WPA2.

Task 2: Plan on paper first

Sample VLSM carve from 192.168.0.0/22 (defend yours): Students 192.168.0.0/24 (120), Staff 192.168.1.0/24 (60), Servers 192.168.2.0/28 (5), Management 192.168.2.16/29, uplinks 192.168.2.24/30 pairs. **Grading reality:** instructors award substantial marks for the plan alone – professionals design before configuring.

Task 3: The keystone configuration – router-on-a-stick

Inter-VLAN routing needs one router interface wearing subinterfaces, one per VLAN:

```
EDGE(config)# interface g0/0
EDGE(config-if)# no shutdown
EDGE(config)# interface g0/0.10
EDGE(config-subif)# encapsulation dot1Q 10
EDGE(config-subif)# ip address 192.168.1.1 255.255.255.0
EDGE(config)# interface g0/0.20
EDGE(config-subif)# encapsulation dot1Q 20
EDGE(config-subif)# ip address 192.168.0.1 255.255.255.0
EDGE(config)# interface g0/0.30
EDGE(config-subif)# encapsulation dot1Q 30
EDGE(config-subif)# ip address 192.168.2.1 255.255.255.240
```

Listing 37: Router-on-a-stick with 802.1Q subinterfaces

The switch side trunks (`switchport mode trunk`) toward the router. Assemble remaining pieces from their weeks: VLANs/trunking/port-security (W5, W12), DHCP pools per VLAN excluding gateways (W8), `ip nat inside source list ... overload` at the edge (W8), student-restriction extended ACL inbound on `g0/0.20` permitting only `eq 80` toward the server host (W12), SSH-only VTYS everywhere (W12), OSPF single-area if you add a second router (W7).

Task 4: The verification gauntlet

Run every row; screenshot passes; investigate failures before decorating anything:

1. Each PC leased a DHCP address in ITS OWN vlan subnet
2. Staff <-> Students ping OK (inter-VLAN via subinterfaces)
3. Student PC browses server web page: HTTP 200
4. Student PC ping to server IP: BLOCKED (ACL, show access -lists hit++)
5. Staff PC pings 8.8.8.8: OK (verify show ip nat translations grew)
6. ssh -l admin <device-ip> works; telnet same IP: REFUSED
7. Rogue laptop on sticky port: blocked (show port-security)
8. Reload EDGE: all configs survive (startup-config discipline)
9. Instructor unplugs ONE trunk; you narrate impact and restore < 3 min

Listing 38: Acceptance checklist – all must PASS

Task 5: Documentation and defense

Submit: topology drawing, addressing table (network/prefix/gateway/VLAN per segment), full device configs with comments, the evidence screenshots, and one page of *design justifications* (why /28 for servers? why ACL direction chosen?). Then defend live: expect your instructor to unplug something, inject a rogue MAC, or demand a policy change mid-demo – graceful recovery under pressure is the true final exam of networking, and after thirteen weeks of building skills layer by layer, you are finally equipped to give it. Congratulations on completing the full arc: signals to services, copper to clouds – the same foundation every network engineer, cloud architect, and security analyst stands on.